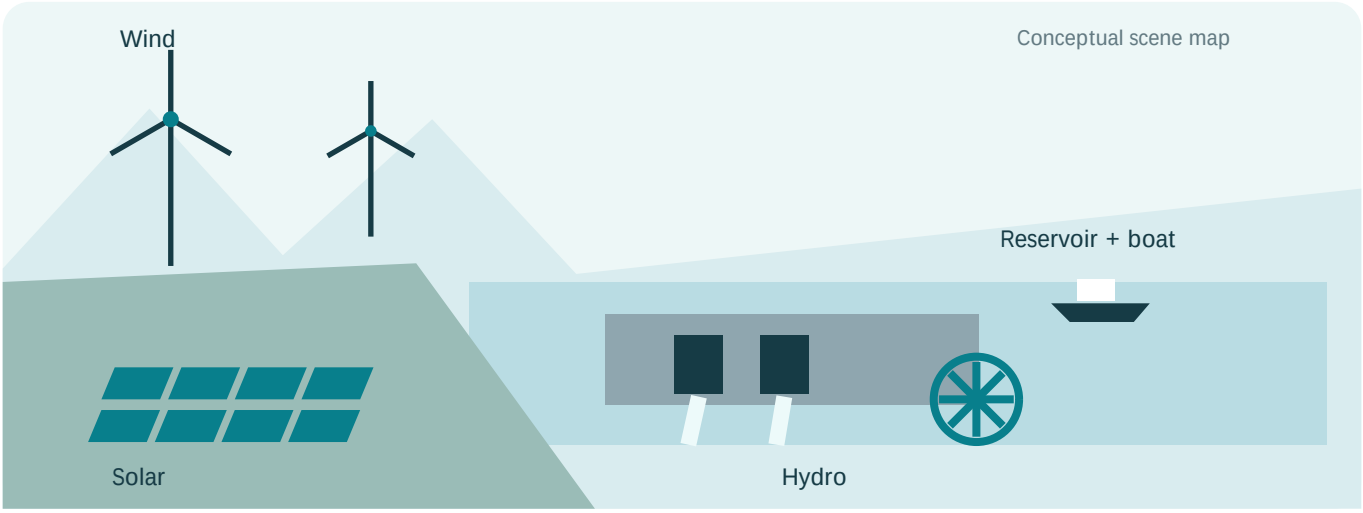
Confluence

A clear plan for your 3D hybrid renewable energy graphics project

Md. Tahmid Islam | Roll 2107107 | CSE 4101



Your final project

An elegant, interactive 3D power plant where wind turbines, a dam, a water wheel, solar panels, clouds, and a boat work together. Every graphics concept has a visible demonstration and a file you can find and understand.

- Lighting and materials
- Translation, rotation, scaling, and hierarchy
- Gouraud and Phong shading
- Continuous motion and user control
- A real ray tracing renderer with shadows and reflections

Implementation roadmap Easy language 8 October 2026

Based on your original two-page proposal, the current Confluence source, the ship simulator, and Siyam s candle project. This document describes work to build; it is not a claim that the planned features are already implemented.

How to use this plan

Read once for the big picture, then build one step at a time.

Pages	What you will find
3 4	Your proposal and your actual starting point
5 6	What to learn from your friends projects
7 8	Where the code should go and how it should connect
9 17	Steps 1 8: foundations, shading, motion, and interaction
18 20	Step 9: actual ray tracing, explained in small parts
21 23	Steps 10 12: atmosphere, feedback, and visual polish
24 25	Steps 13 14: verification, presentation, and handover
26	Simple definitions, optional additions, and sources

What the completed application should feel like

The first view clearly shows solar panels in front, two turbines on the hillside, the dam and water wheel in the middle, and the reservoir with a moving boat behind them. The scene has a consistent scale, readable lighting, gentle animation, and a small information panel.

You can explore the scene, change wind and sunlight, open the gate, steer the boat, pause time, and switch shading methods. A separate comparison view lets you explain the graphics calculations without searching around the landscape.

> **The finish line**
>
> You can demonstrate every required feature live, explain the important calculation in simple words, open the file that contains it, and show evidence that it works. A large feature count by itself does not establish project quality.

Use three milestones

- Milestone A after Step 4: a clear lighting and shading demonstration.
- Milestone B after Step 8: all proposal objects move and respond to input.
- Milestone C after Step 14: ray tracing, polish, documented checks, and a rehearsed presentation.

The step order is a dependency order, not a promised timetable. Finish each visible checkpoint before adding the next feature. Keep a working build at every milestone.

Keep your original proposal

The renewable energy scene remains the centre of the project.

Your proposal is titled 3D Hybrid Renewable Energy Power Plant Simulation Using OpenGL. It explicitly focuses on graphics demonstrations rather than exact electrical generation or complex water physics. Keep that purpose, title, and scene.

Original promise	How the final version fulfils it
Dam, gate, and water outlet	A gate moves vertically on command; visible flow responds to its opening.
Two wind turbines	Two separate rotors turn around their hubs and respond to wind strength.
Solar panel array	Panels retain the foreground layout and show clear surface highlights; optional tilt control has a real hinge.
Reservoir and moving boat	Animated water, a boat patrol, manual steering, and slight wave motion.
Hydro water wheel	The wheel rotates beside the outlet; its speed follows available flow.
Moving cloud and environment	Clouds drift and wrap; terrain, sky, and supporting structures remain coherent.
Phong sunlight and specular effects	Sunlight, material properties, and visible highlights on water, panels, and metal.
C/C++, OpenGL, time, camera	The same project stack, with a timed update loop and explicit camera/input modules.

Additions that fit the same proposal

- Gouraud/Phong comparison, lighting controls, and graphics debugging.
- Direct plant controls, boat steering, and named camera views.
- A CPU ray renderer of the same plant, with camera rays, shadows, and reflections.
- Simple day/night and weather presets, particles, and labelled educational output indicators.

> **Preserve the proposal file**
>
> Treat this roadmap as a companion document. Keep the original proposal PDF unchanged and describe new work as extensions of its renewable energy graphics scene.

What you already have

Source inspection of Confluence; the application was not run for this audit.

Feature	Evidence and current state
3D scene and separate object files	Dam, turbines, panels, boat, water, wheel, terrain, trees, rocks, fence, clouds, and sky already have source files.
Camera interaction	main.cpp registers left-drag orbit, wheel zoom, R reset, and Escape exit.
Transformations and hierarchy	Turbine.cpp builds tower nacelle hub blade frames. Dam.cpp accepts a gate opening parameter.
Lighting and materials	The shaders contain ambient, diffuse, specular, attenuation, and Phong/Blinn formulas. main.cpp currently supplies two directional lights.
Gouraud/Phong paths	phong.vert calculates vertex lighting; phong.frag uses either interpolated colour or per-fragment lighting. Keys 1/2/3 select the modes.
Static poses	Scene.cpp sends fixed turbine angles 0.35 and 1.35, wheel angle 0.3, and Layout::GATE_OPENING = 0.45.
Water, clouds, and boat	Geometry and poses are created at startup. No changing simulation time is supplied to their drawing. The wake and mist are currently static geometry.
Ray tracing and plant input	No ray renderer, wind control, gate input, or boat steering was found in the Confluence source.
Screenshot support	Screenshot.cpp and --screenshot already provide a BMP capture path. Keep and extend this.

The next work is very specific

Build the timing/state layer, connect the existing object parameters to it, add direct input, then add the comparison gallery and ray renderer. Reuse the objects you have already built.

> **Fix while extending**
>
> The lighting function is duplicated in the vertex and fragment shaders. Share one source block. The current Flat branch uses a face normal but still evaluates lighting per fragment; document this distinction or implement constant colour per face for a strict Flat demonstration.

What the ship project teaches you

Transfer its useful graphics ideas into the power plant.

Observed in the ship source	Equivalent for Confluence
Hierarchical ship parts	Tower nacelle rotor blades; dam gate slot moving gate.
Sailing and steering input	Manual reservoir boat control, with a patrol/manual switch.
Moving sea and ship response	A gentler reservoir wave field that also drives boat height and tilt.
Shared lighting source	One lighting block used by both Gouraud and Phong shader stages.
Lighting and shading keys	Same scene, light, material, and camera; change only the demonstrated method.
Normals, wireframe, and detail levels	A small gallery plus a turbine tower or water patch with visible triangle counts.
Time, weather, and camera presets	Day/sunset/night and clear/cloudy/rain, plus useful plant inspection views.
Particle pools	A reusable pool for spray, rain, gate foam, and boat wake particles.
HUD and screenshots	Compact plant controls and matching screenshots for your report.
Measured checks	Record what changed, why it is correct, and how it was checked.

Its ray tracing has a precise scope

The inspected basic.frag casts one sun visibility ray from each eligible ocean fragment against at most 16 proxy spheres. A hit reduces direct sunlight on the water. The main scene still uses rasterization; its planar reflection is also rasterized. This is a real but limited hybrid ray tracing feature.

Your required ray renderer will trace camera rays into a snapshot of the plant geometry and add shadow and reflection rays. An optional live shadow-ray mode can later bring a similar effect into your interactive scene.

> **Keep the scene focused**
>
> The ship s trade, combat, ports, and missions belong to its naval theme. Your equivalent depth comes from connected wind, water, sunlight, plant controls, and clear graphics demonstrations.

Reference evidence: src/main.cpp, Lighting.h, Waves.h, CameraPresets.h, Particles.h, shaders/basic.frag, README.md, and report/main.tex in the ship project. Some earlier documentation describes an older stage; the inspected source determines the observations above.

What the candle project teaches you

Small, understandable systems can show many graphics ideas.

Observed in Siyam s project	Useful adaptation for your scene
One file pair for each object	Keep one understandable module for each plant object and one for each graphics system.
Explicit transform helpers	Explain translation, rotation, scale, pivot motion, and matrix order in Transform.*.
Shear and reflection support	If your course expects them, show gentle canopy/reed bending and mirrored model geometry in the gallery.
Directional and point lighting	Use sunlight plus a real dam lamp or movable inspection light.
Perspective/orthographic switch	Show depth naturally in perspective and compare proportions in orthographic mode.
Matrix descriptions and HUD	A debug panel can name the selected object, show its values, and optionally print its matrices.
Window wind flame relationship	Wind rotor speed; gate flow wheel speed; sunlight panel indicator.

Read its current version accurately

Siyam s main.cpp describes Step 1: scene objects. It switches between two posed configurations; the provided code does not yet continuously animate the flame and curtain. Its Shading.* computes lighting on the CPU and subdivides triangles to approximate per-pixel Phong results. Your existing GLSL fragment shader is the appropriate path for your Phong demonstration.

Make the graphics visible to the teacher

Required concept	A clear live proof
Lighting	Turn diffuse or specular off; move the light and observe the result.
Transformation	Show a blade rotating about its hub and a gate sliding in its own frame.
Gouraud and Phong	A coarse shiny mesh shows a highlight difference under identical conditions.
Motion	Pause six named moving components; resume them smoothly.
Interaction	Operate the gate, change wind, and steer the boat with user input.
Ray tracing	Show the same camera snapshot with shadow/reflection rays enabled and disabled.

> **These are project recommendations**
>
> Your request and proposal define the required features. No separate marking rubric was found in the workspace. This plan makes the concepts easy to assess; it cannot guarantee a particular grade.

A file structure you can understand

Keep the existing layout and add systems gradually.

```
Confluence/
  CMakeLists.txt                build the C++ application
  include/ + src/glad.c         existing OpenGL loader
  shaders/
    phong.vert / phong.frag     existing shading paths
    lighting_common.glsl        NEW: shared lighting equations
    water.vert / water.frag     NEW: water motion and appearance
    screen.vert / screen.frag   NEW: display ray-rendered pixels
    sky.vert / sky.frag         existing sky
  src/
    main.cpp                    setup and the small main loop
    core/
      Camera.*  Shader.*  Mesh.* Render.* Screenshot.*
      Light.h  Material.h       keep existing reusable data
      Transform.*               NEW: documented matrix helpers
      SceneState.h  Clock.*     NEW: controls, poses, and time
      DrawItem.h                NEW: shared scene draw records
    objects/
      Scene.*  Layout.h         existing assembly and placement
      Turbine.*  Dam.*  WaterWheel.*  SolarArray.*
      Water.*  Cascade.*  Boat.* Cloud.*  Sky.*
      Terrain.*  Trees.*  Rocks.* Fence.*
    input/  Input.*  Picking.*   NEW: keys, mouse, selection
    systems/
      Animation.*  Waves.*      NEW: motion and water maths
      Environment.*             NEW: sun, sky, weather
      Particles.*  EnergyModel.* NEW: effects and indicators
    rendering/
      RasterRenderer.*          NEW: draw records with OpenGL
      Ray.h  RayTracer.*        NEW: camera/shadow/reflection rays
      SceneSnapshot.*  BVH.*    NEW: frozen triangles and search
    ui/  Hud.*  DemoGallery.*   NEW: controls and comparisons
  docs/                         explanations and demo checklist
  captures/                     measured screenshot evidence
  build/                        generated compiler output
```

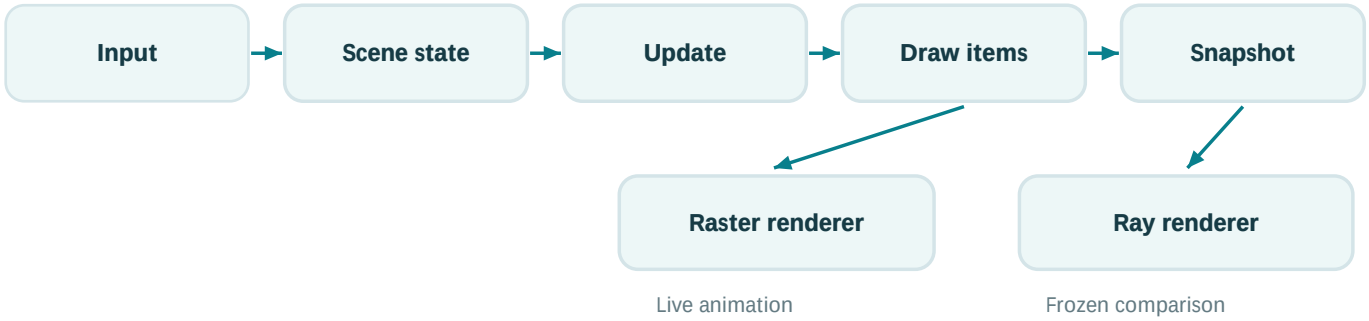
NEW means planned, not present in the current source. .* means a .h declaration and .cpp implementation where useful. Small data-only types can stay in a header. Avoid splitting a short helper into several unnecessary files.

A readable main.cpp

main.cpp should initialise the application, poll events, read input, update state, select a renderer, draw the scene/UI, and shut down. Geometry, lighting equations, ray intersection code, and controls belong in their own modules.

How the files should connect

One scene state supplies both renderers.



Module	Its single job
SceneState.h	Store wind, gate target/current opening, rotor angles, boat pose, panel tilt, simulation time, and mode choices.
Input.* / Picking.*	Turn a key, mouse gesture, or selected-object edit into an action.
Animation.* / Waves.*	Advance motion from elapsed seconds; provide shared water height and normals.
Environment.* / EnergyModel.*	Derive light/weather settings and labelled educational indicators.
Object files / Scene.*	Build local meshes once, calculate current part transforms, and collect DrawItem records.
DrawItem.h	Record mesh identity, model matrix, material, object ID, and rendering/shadow flags.
RasterRenderer.*	Send draw records to the existing OpenGL shaders.
SceneSnapshot.* / BVH.*	Copy a frozen view of the same records into ray-testable triangles and bounding boxes.
RayTracer.* / Hud.*	Trace/display pixels; show controls, progress, values, and explanations.

Rules that make later learning easier

- Use clear names and explicit units: windStrength, gateOpening, rotorAngleRad, boatSpeedUnitsPerSecond.
- Keep layout numbers in Layout.h and tuning values beside their system. Keep mutable state out of constants.
- Every new step gets a note: what it does, why it exists, the important formula, files, controls, and checkpoint.
- Reuse material/light data across renderers. Keep CPU and GLSL lighting equations equivalent and compare their results on a tiny scene.
- Keep static geometry cached. A moving part needs its own transform or deformable mesh; do not merge it permanently into a fixed structure.

Step 1 Establish time and state

The first missing system is a changing simulation.

Build

Save a working scene screenshot and record the current build/run instructions. Keep the existing MinGW/MSYS2 setup unless there is a concrete reason to change it.

Add Clock.* and SceneState.h. Separate real frame time from simulation time.

Restructure the loop into input update render. Add Scene::update or an Animation system that updates state before drawing.

Use a pause flag, time scale, and reset action. Camera navigation and UI must still work while simulation time is paused.

The simple calculation

```
realDelta = currentRealTime - previousRealTime
simulationDelta = paused ? 0 : realDelta * timeScale
simulationTime += simulationDelta
angle += angularSpeed * simulationDelta
position += velocity * simulationDelta
```

Delta means seconds since the last frame. Use realDelta for the camera and simulationDelta for plant motion. Clamp an unusually large delta after a stall, or use bounded update steps, so the boat and gates do not jump.

State versus drawing

Wind strength, gate opening, rotor angles, and boat position are changing numbers. Geometry construction creates the parts. Drawing reads the current numbers to place them. A draw call should not silently advance animation.

New or changed files	Purpose
core/Clock.*	Elapsed seconds, pause, and time scale.
core/SceneState.h	One place for mutable settings and poses.
systems/Animation.*	Motion updates.
main.cpp / objects/Scene.*	Call input and update before render.

Done when

A test rotor moves steadily. Pause stops it, resume continues without a jump, and the camera still moves. Changing frame rate does not materially change distance travelled in ten seconds.

Step 2 Prove the transformations

Make it easy to show why each object moves correctly.

Concept in easy words	Where to demonstrate it
Translation move an object	Gate moves upward; boat and cloud move through the scene.
Rotation turn around an axis	Blades turn around the rotor axis; the wheel turns around its axle.
Scaling change dimensions	One unit cube/cylinder makes many sizes of supports, towers, and panels.
Composition combine moves	Place, orient, and size each mesh with its model matrix.
Hierarchy children follow parents	The turbine root carries its nacelle, hub, and blades together.
Pivot use a chosen turning point	Rotor hub and panel hinge stay fixed while their parts rotate.

Explain matrix order with one example

```
worldPoint = Model * localPoint
Model = Translate * Rotate * Scale
clipPoint = Projection * View * Model * localPoint
pivotRotation = Translate(pivot) * Rotate * Translate(-pivot)
```

With GLM s column-vector convention, the rightmost operation happens first. Apply size to an individual part at drawing time. Keep parent frames separate from part-size scaling so a stretched tower does not stretch its children.

Build the supporting proof views

Add Transform.* with short comments and explicit radians/degrees.

Show wireframe, normal colours, and low/medium/high mesh detail on a small gallery object. Detail changes triangle count; scaling changes size.

Check inverse-transpose normals under nonuniform scaling. Avoid zero scale, which makes the inverse undefined.

For extra course coverage, show shear on a canopy/reed and geometric reflection on mirrored model parts. Reflection reverses triangle winding; account for this when culling is enabled.

Files: core/Transform.*, core/Mesh.*, core/Render.*, ui/DemoGallery.*, and the existing object transform code.

> **Done when**
>
> A rotor turns in place, a gate follows its local slot, and moving a turbine root carries all its children. A gallery toggle compares correct and reordered matrices, and you can explain the difference.

Step 3 Make lighting readable

Shapes, materials, and light controls should tell a clear story.

Build three useful light behaviours

Directional sunlight: one direction across the whole plant. Moving the sun changes the illuminated sides.

A local point light: a dam lamp or movable inspection light. Its contribution becomes weaker with distance.

A small global ambient term: shaded surfaces remain readable. Keep it low enough that diffuse lighting still has a visible effect.

Lighting term	What the viewer sees
Ambient	A basic amount of background light on the material.
Diffuse	A surface is brighter when its normal faces the light.
Specular	A shiny highlight depends on surface, light, and camera directions.
Emission	A lamp surface looks bright by itself. A separate point light is needed to illuminate nearby objects.
Attenuation	A point light contributes less as distance increases.

Use distinct, believable materials

Concrete has weak broad highlights; painted turbine parts have moderate highlights; steel gates and panel glass have tighter highlights; water has a visible glint. Keep existing vertex colours, and give important material groups separate draw records instead of one generic glossy material for every part.

Add controls and share the formula

K cycles ambient-only, diffuse-only, specular-only, and all terms. L cycles light combinations. B switches the Phong and Blinn specular formulas. Move the inspection light through UI controls. Extract duplicated GLSL lighting into lighting_common.glsl and inject it after #version; standard GLSL 3.30 does not provide #include processing.

Files: core/Light.h, core/Material.h, core/Shader.*, shaders/lighting_common.glsl, phong.vert, phong.frag, and systems/Environment.*.

> **Done when**
>
> You can isolate each term, move a point light, and see its falloff. A highlight moves with the camera. Disabling specular removes highlights. The sun/fill balance does not wash out the objects.

Step 4 Show Gouraud and Phong clearly

The comparison changes the shading method alone.

Mode	What is calculated
Flat	For the strict teaching version, use one face normal and calculate one colour per triangle.
Gouraud	Calculate lighting at each vertex, then blend those colours across the triangle.
Phong	Blend vertex normals, normalise the result, and calculate lighting for each fragment.

The Phong lighting model is an equation for surface brightness. Phong shading evaluates lighting using an interpolated normal at each fragment. Gouraud can use the same Phong lighting equation at the vertices. The equation and the shading method have different jobs.

Create a small comparison gallery

- Use a coarse shiny sphere/cylinder and a low-detail flat water/panel patch.
- Provide 1/2/3 for Flat/Gouraud/Phong and B for the specular formula.
- Hold camera, mesh, material, light, and exposure constant during comparison.
- Place a narrow highlight between vertices. Gouraud can miss or weaken it because no vertex samples its peak; Phong can reveal it inside the triangle.
- Show three detail levels with triangle counts and wireframe. Higher detail improves sampling; it does not replace the Phong path.

Handle the existing Flat branch accurately

Your shader reconstructs a face normal using screen-space derivatives, then evaluates lighting per fragment. This gives visible facets, but a point light can still vary across one triangle. For strict constant-colour Flat shading in the gallery, calculate face-centre lighting once and supply the same colour to all three corners. Check reconstructed normal orientation against the outward normal.

Keep comparisons reproducible

Save a camera preset, paused time, light settings, mesh detail, and material preset for each demonstration. Disable ray effects and weather overlays during the pure Gouraud/Phong comparison. Label modes and capture matching screenshots.

Files: ui/DemoGallery.*, shaders/phong.vert, phong.frag, lighting_common.glsl, and core/Mesh.*.

> **Done when Milestone A**
>
> The same coarse shiny object shows a clear highlight difference between Gouraud and Phong. You can point to the vertex and fragment calculations in the code.

Step 5 Animate turbines and clouds

The first continuous motion uses the objects you already have.

Wind turbines

Replace the fixed rotor angles in Scene.cpp with two angle values in SceneState.

Use one wind setting to drive both turbines, while allowing a small configured speed difference. Each turbine remains an independent object.

Update angle by speed simulationDelta. Keep units in radians. Smooth changes in speed and wrap large angles without changing the visible pose.

Keep towers and nacelles fixed while blades turn about the hub. Optional yaw adjustment can rotate the nacelle toward the chosen wind direction.

```
rotorAngle += rotorSpeedFromWind * simulationDelta
```

When wind changes, integrate the new angular speed. Recalculating angle as currentWind totalTime would abruptly jump the rotor. The same principle applies to a cloud speed or patrol speed changed during the run.

Clouds

Give each cloud an offset and velocity; translate the existing cloud root each update.

Wrap cloud movement beyond the visible sky boundary. Avoid a visible teleport in the centre of the scene.

Use a fixed random seed for cloud shape so repeated runs and screenshots are comparable.

Represent overcast conditions using an explicit cloud-cover value at first; detailed cloud-to-panel shadow geometry can be added later.

User input

Q/E decrease/increase wind. The HUD shows the value. A small wind arrow explains its direction. Wind strength should visibly change rotor speed and later change the educational wind indicator.

Files: objects/Turbine.*, objects/Cloud.*, objects/Scene.*, core/SceneState.h, systems/Animation.*, and input/Input.*.

> **Done when**
>
> Both rotors turn continuously, their towers stay fixed, clouds drift, and wind control changes speed smoothly. Pause stops all of these motions and resume produces no discontinuity.

Step 6 Connect the gate, flow, and wheel

One user action should cause several understandable changes.

Dam gate

Replace Layout::GATE_OPENING with mutable gateOpening and gateTarget values between 0 and 1.

G toggles the gate target between shut and open. F/H gradually lower/raise the target when held.

Move the current opening toward the target over time, with a maximum speed. Keep it within the slot.

Supply the same current opening to the gate transform, spillway appearance, spray emitter, wheel speed, and educational hydro indicator.

Water outlet and hydro wheel

Use a simple relationship such as flowStrength = gateOpening. As flow increases, water sheets become stronger, foam/spray increase, and the water wheel speeds up. Integrate wheel angle using its current speed. When the gate closes, reduce the linked outlet flow and smoothly slow the wheel.

The existing wheel-channel fall is separate from the two spillways. Decide explicitly that the main gate also controls the shared hydro supply, or give that channel its own valve. Then make every channel, spray effect, and wheel agree with that choice. Never keep a static waterfall visible under a closed supply.

Demonstrate the hierarchy

The gate frame is damRoot gateSlot verticalTranslation. The wheel frame is damRoot wheelMount wheelRotation. Rotate the dam root in a gallery test and show that attached parts keep their correct positions.

Show six moving components together

Add a labelled automatic demonstration mode that slowly cycles the gate. Normal operating mode is manual. Any gate input exits the automatic cycle. With the boat patrol enabled, the two turbines, cloud, gate, boat, and wheel can all move at the same time as promised.

Files: objects/Dam.*, WaterWheel.*, Cascade.*, Water.*, Scene.*, systems/Animation.*, core/SceneState.h, input/Input.*, and ui/Hud.*.

> **Done when**
>
> Opening the gate visibly raises it, increases outlet flow, increases spray, and speeds the wheel. Closing the supply removes linked falling water and slows the wheel. You can show the attached parts following the dam frame.

Step 7 Make water and the boat move together

One wave description should drive both systems.

Animated water

Start with two or three gentle travelling sine waves. Add their heights to the reservoir grid vertices and calculate normals from the slope, so light glints move with the surface. Keep the lower river/outlet appearance stronger and more turbulent.

```
height(x,z,time) = meanLevel + sum of travelling waves
```

Store wave directions, amplitude, wavelength, and speed in Waves.*. Supply the same parameters to GLSL, the boat update, and the ray snapshot. Do not maintain unrelated copies of the water formula. Avoid rebuilding the whole dense river mesh every frame; use shader deformation or targeted dynamic buffers.

Boat patrol and manual steering

- Add a closed patrol path inside the reservoir; update path progress with elapsed seconds. The boat faces the path direction.
- V switches between patrol and manual steering. In manual mode W/S change forward speed and A/D turn.
- Sample water under bow, stern, left, and right. Their average sets height; their difference sets small pitch and roll.
- Limit motion to navigable reservoir water. Use the dam-local bounds and shoreline tests rather than letting the boat pass through land or the dam.
- Keep wake particles near the water and emit more when moving faster. At rest, a long fixed wake should disappear.

Preserve moving parts

Boat.cpp currently includes wake geometry in the boat mesh. Move the wake into a separate effect so it can trail and fade. SolarArray.cpp currently bakes its 30-degree tilt into a combined mesh. If adding adjustable tilt, split moving glass/frame groups from fixed support legs and rotate the panels around their hinge.

Files: systems/Waves.*, Animation.*, Particles.*, objects/Water.*, Boat.*, SolarArray.*, shaders/water.vert, water.frag, and Scene.*.

> **Done when**
>
> The boat travels, turns, bobs, and tilts with the visible water. It can be steered without crossing the shore or dam. Wake follows motion, and a paused screenshot captures the same water height in both renderers.

Step 8 Give the user meaningful control

Interaction means input changes scene behaviour.

Core interaction

Provide working controls for wind, gate opening, boat speed/heading, pause/resume, simulation speed, shading, and lights.

Add named camera views: overview, turbine, dam/wheel, solar array, and boat. A follow-boat camera remains optional.

Add perspective/orthographic projection, orbit/zoom, and simple camera limits. Resize the window without stretching the scene.

Allow a right click to select a turbine, gate, panel array, or boat. Highlight the selected object and show its relevant settings. Tab cycles the same selectable objects for keyboard access.

Picking in simple words

Convert the mouse location into a ray from the camera. Test it against selectable objects bounding boxes or meshes, and select the nearest positive hit. A picking ray chooses an object; it does not by itself render a ray-traced image. Convert window coordinates to framebuffer coordinates correctly on scaled displays.

Handle keys correctly

A toggle acts once on the key press: gate toggle, pause, shading, and modes. A held key changes a value continuously: wind, gradual gate adjustment, or steering. Do not use operating-system key-repeat timing to drive movement. GLFW s input guide describes event callbacks and polling [4].

Make behaviour easy to inspect

Show the active mode and current values immediately. Show target and current gate opening separately while it is moving.

Clamp values to useful ranges. Show a clear reason when the boat reaches the shore.

When a UI field is active, do not also steer the boat with the same key.

Keep the camera responsive during pause and ray rendering. Provide reset-camera and reset-scene actions separately.

Files: input/Input.*, Picking.*, core/Camera.*, SceneState.h, ui/Hud.*, DemoGallery.*, and main.cpp.

> **Done when Milestone B**
>
> Without editing code, you can change wind, operate the gate, steer the boat, pause six components, inspect each plant area, and compare shading. Each control has visible feedback.

Suggested controls

A proposed mapping; your help panel must show the final mapping.

Input	Action
Left drag / mouse wheel	Orbit / zoom the camera.
Right click / Tab	Select the nearest object / cycle selectable objects.
F1 / F2 / F3 / F4 / F5	Overview / turbine / dam+wheel / panels / boat view.
R / Enter	Reset camera / reset the whole scene.
P	Perspective or orthographic projection.
Space	Pause or resume simulation; camera remains usable.
[/]	Decrease / increase simulation time scale.
Q / E	Decrease / increase wind strength while held.
G	Toggle gate target open/shut.
F / H	Lower / raise gate target while held.
J	Toggle labelled automatic gate demonstration.
V	Switch the boat between patrol and manual control.
W / S; A / D	Manual boat mode: W accelerates; S brakes/reverses; A/D turn left/right.
Z / C	Decrease / increase selected panel tilt, if implemented.
1 / 2 / 3	Flat / Gouraud / Phong shading.
B / K / L	Specular formula / lighting terms / active light combination.
T / Y	Cycle day, sunset, night / clear, cloudy, rain presets.
X / N / + / -	Wireframe / normals / gallery mesh detail up/down.
F6	Enter or leave the graphics comparison gallery.
0	Enter ray snapshot mode or return to live raster view.
F7	Re-render the current frozen ray snapshot.
F8 / F9	Toggle ray shadows / ray reflections and re-render.
F10	Match raster/ray views at the same frozen pose and camera.
F12	Save the current image and its settings.
Backspace / F11	Show/hide the information panel / help.
Escape	Cancel ray work first; otherwise quit the application.

Controls panel

Display the few actions relevant to the selected object. Put the full list behind Help. Add UI sliders for wind, sun angle/intensity, gate target, panel tilt, point-light position, and ray image quality as those features are built.

Step 9 Implement a real ray renderer

A separate renderer uses the same plant scene.

Choose an understandable implementation

Keep OpenGL for the fast interactive scene. Add a small C++ CPU ray renderer for a frozen snapshot of that scene. OpenGL displays the resulting pixel image in a texture. This works without requiring an RTX-specific API or changing the existing OpenGL 3.3 rendering foundation.

Required part	What it must do
Camera rays	Send rays through image pixels to find visible plant surfaces.
Geometry intersection	Find the nearest hit against actual exported triangles. Begin with sphere tests in a tiny practice scene.
Surface lighting	Use the scene s material colours, vertex colours, normals, sun, and point lights.
Shadow rays	Check whether another surface blocks a light from the hit point.
Reflection rays	Bounce from reflective water, panel surfaces, or polished metal and trace the reflected view.
Finite work	Use a maximum reflection depth and a small image size for the first version.
User interaction	Start/cancel rendering, show progress, change quality, and compare the same snapshot.

What the ray snapshot should contain

Include the main opaque geometry: dam, gates, towers, blades, wheel, panels, terrain, boat, rocks, fences, and trees. Include a frozen reflective water surface. Preserve the current transforms and wave heights. Use the same material groups and base colours as the raster view.

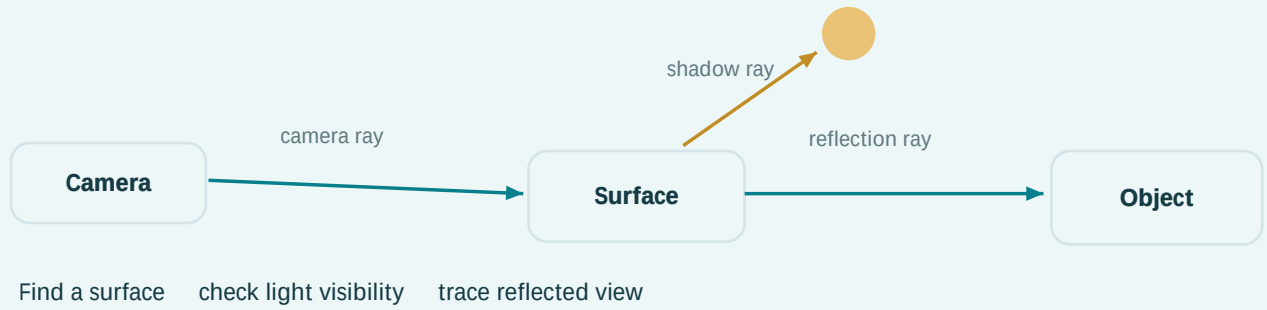
For the first version, treat panel glass as an opaque reflective surface. Use a sky background; rain, mist, and complex cloud transparency may be omitted with a visible scope note. These effects do not need to delay camera, shadow, and reflection rays.

> **Precise claim for the presentation**
>
> Call it a CPU ray-traced snapshot of the plant, with hard shadows and bounded mirror reflections. Describe its treatment of water, transparent effects, and recursion. A shiny highlight or a mirrored raster pass alone is not evidence of ray tracing.

This renderer design is a recommendation for your project. Camera/visibility concepts are supported by PBRT [1], triangle intersection by [2], and acceleration by [3]. The exact scope above is chosen to fit your existing scene.

Step 9 continued Follow a ray

Build and check each small part before using the whole scene.



1. Make a Ray with origin, normalised direction, and valid distance limits. A point along it is origin + distance direction.

2. Build a tiny practice scene with a sphere, a triangle, a ground plane, and one light. Render a small image before exporting the plant.

3. Generate a camera ray through each pixel centre. Use the same camera, aspect ratio, and projection as the raster view. Perspective rays fan out; orthographic rays are parallel. Honour the near/far clip limits.

4. Find the nearest positive intersection. If there is no hit, use the sky colour. A triangle hit provides barycentric weights: three numbers that blend its corner normals and colours.

5. Apply the scene s ambient, diffuse, and specular lighting at the hit. Normalise the blended shading normal before lighting.

6. Send a shadow ray toward each light. If an object blocks it, suppress that light s direct diffuse/specular terms. Keep ambient and emission. A point-light ray stops at the light; a sun ray has no finite light endpoint.

7. For a reflective material, calculate the reflected direction and trace another ray. Mix local lighting and reflected colour with a bounded reflectivity value.

8. Stop after two reflection bounces initially. Offset secondary-ray origins a small, scale-aware distance using the geometric normal, toward the outgoing side, to avoid hitting the same surface again.

9. Store the pixel colour, apply the same display/gamma convention used for comparison, and show completed tiles in the OpenGL window.

Done when

In the tiny scene, one object hides another, a blocker casts a shadow, and a reflective surface shows another object. Then the same tests work in the plant view. Toggling shadow/reflection rays changes the image for the expected reason.

Step 9 continued Data, speed, and proof

Make the ray renderer correct and responsive.

Use a frozen scene snapshot

Keep CPU copies of mesh vertices and indices. Collect DrawItem records from the same scene assembly used by OpenGL. At capture time, transform positions into world space, transform normals correctly, preserve vertex colours/material IDs, and evaluate water deformation at the frozen simulation time. Existing batched meshes already contain local part transforms; do not apply them a second time.

Copy the camera, lights, settings, and triangles into an owned snapshot. Entering ray mode freezes the comparison pose. A camera or setting change marks the old image as stale and starts a new requested snapshot. Never mix tiles from different snapshots.

Use a bounding volume hierarchy (BVH)

A BVH groups triangles inside boxes. A ray that misses a group s box skips that group. Start with a simple tree that splits triangles by their centre along the longest axis, with a few triangles per leaf. Build it once per snapshot. Compare its hits with a slow all-triangle search on a tiny scene before relying on it for the dense plant [3].

Render in small pieces

Begin around 320 180 pixels, one sample per pixel, two reflection bounces. Raise quality only after measuring actual render time.

Process small tiles or a short work budget each frame; continue polling events and updating the progress panel. Cancel immediately when requested.

A later worker thread may compute pixels; OpenGL uploads remain on the context-owning thread.

Save the image, render time, resolution, triangle count, maximum depth, camera, and snapshot settings. A snapshot render time is different from interactive FPS.

Files	Responsibility
Ray.h / RayTracer.*	Ray data, intersection, lighting, shadow/reflection recursion, and tiled image rendering.
SceneSnapshot.*	Owned camera/light/material/triangle data at one pose and time.
BVH.*	Bounding boxes, tree construction, and ray traversal.
screen.vert / screen.frag	Display the computed image as an OpenGL texture.

Done when

BVH and direct search agree on a fixed test set; moving a gate changes its ray silhouette and shadow; reflection depth terminates; a camera change invalidates old work; render progress and cancellation remain usable.

Step 10 Add atmosphere and particles

Weather should support the plant s graphics and behaviour.

Keep the first environment system small

Preset or effect	Visible change
Day / sunset / night	Change sun direction, intensity, colour, sky gradient, and fog smoothly. Activate dam lamps at night.
Clear / cloudy / rain	Change cloud cover, brightness, wind, and light rain. Avoid instant random changes while demonstrating a formula.
Spray and mist	Particles appear near active outlet flow, move with wind, and fade.
Boat wake	Small foam particles follow the moving boat and disappear with age.
Wind hint	A small wind arrow and a few gentle environmental movements show direction.

Use one particle system

Each particle stores position, velocity, age, lifetime, size, and colour. Reuse particles from a bounded pool rather than allocating new ones forever. Update position using elapsed seconds, fade with age, and reuse an expired slot. Emission rate follows flow for spray, speed for wake, and rain intensity for rain.

Draw transparency carefully

Draw opaque surfaces first. Draw transparent effects afterward with blending. Keep depth testing on, usually disable depth writing for fading particles, and restore it after the effect pass. Sort ordinary alpha particles roughly back to front; additive glow is a different blend mode. Avoid large visible intersections of mist spheres.

Make night lighting real

An emissive lamp mesh shows where the light comes from. A point light must separately illuminate nearby dam geometry. Keep night readable with dim ambient light. A sun below the horizon should not continue adding daytime direct light.

Files: systems/Environment.*, Particles.*, objects/Sky.*, Cloud.*, Water.*, Boat.*, core/Light.h, and the relevant shaders.

> **Done when**
>
> The scene shows clear day, sunset, and night differences. Night lamps affect nearby surfaces. Spray follows flow, wake follows boat speed, particles are bounded, and pause freezes the effects.

Step 11 Add simple energy feedback

Give the scene a clear renewable energy identity.

Show cause and effect

Add three small bars for wind, solar, and hydro contribution. When a user changes wind, sunlight/panel tilt, or gate opening, the relevant animation and indicator should respond. Show an optional total bar and a target line for a simple balancing exercise.

Indicator	A useful simplified rule
Wind	Use a clamped wind setting in a named curve, such as a normalised cubic curve below a configured cap. Use the same wind setting that drives the rotors.
Solar	Use daylight cloudFactor max(dot(panelNormal, directionToSun), 0). At night the contribution is zero.
Hydro	Use normalised flowStrength normalisedHead. Keep head fixed in the first version; connect flow to the current gate opening.
Total	Combine contributions with clearly named capacities or weights. Use this total for the target line.

Keep the explanation honest and simple

Label the panel Illustrative output or Normalised contribution. Use bars, percentages, or relative units. Do not label arbitrary values as measured kW. These rules are visual educational models chosen for the project; they do not simulate a real electrical grid.

A short interaction example

1. Start with weak wind and a partly open gate. The wind bar is low and the hydro bar is moderate.
2. Increase wind: the turbines turn faster and the wind bar rises.
3. Choose night: the solar bar falls to zero and the dam lamps become useful.
4. Open the gate: hydro rises, the wheel accelerates, and the outlet becomes stronger.
5. Use a labelled reset preset to repeat the same demonstration.

Files: systems/EnergyModel.*, systems/Environment.*, core/SceneState.h, objects/SolarArray.*, and ui/Hud.*. Keep these indicator rules out of drawing and input callbacks.

> **Done when**
>
> Every bar follows the same setting used by the visible scene. You can explain each rule in one sentence. Closing a gate or choosing night cannot leave the corresponding output at an unrelated value.

Step 12 Make it elegant and easy to use

Polish should improve readability and consistency.

Scene composition

Keep the proposal s visual hierarchy: foreground panels, hillside turbines, central dam/wheel, and a boat on the reservoir.

Use believable relative sizes and stable contact with the terrain. Check support legs, boat waterline, gate clearance, and rotor spacing.

Choose a restrained palette: blue water/panels, pale concrete/turbines, darker vegetation, and small warm lights.

Tune highlights rather than making every surface equally glossy. Use gentle fog to separate distant terrain from the main objects.

Animate smoothly at a sensible scale: readable gate travel, calm boat motion, slow clouds, and controlled particle density.

Interface

Use a compact panel with aligned labels, current values, short hints, and an obvious selected-object name. Give pause, shading, weather, and ray rendering clear status labels. Keep controls readable at different window sizes. Show full help on demand and allow a clean screenshot view.

Views that make presentation reliable

View	Why it exists
Overview	Shows the original proposal scene and its six moving components.
Turbine close-up	Shows the hub pivot and parent-child transformations.
Dam and wheel	Shows gate input, water flow, and connected motion.
Solar close-up	Shows panel material highlights and optional tilt.
Graphics gallery	Shows shading, normals, mesh detail, and lighting terms.
Frozen ray comparison	Shows actual shadow/reflection ray effects at one camera and time.

Small details worth doing

Add a stable window title, clear startup errors, a sensible default camera, reset presets, and a screenshot action that also records settings. Check both normal operation and the teaching view. Keep the overview uncluttered.

> **Done when**
>
> The first screen communicates the plant clearly. A new viewer can find the controls, operate an object, and understand the visible response. Each demonstration view can be reached in one action.

Step 13 Check correctness and performance

Collect evidence for the remaining technical risks.

Check	Expected evidence
Frame-rate independence	Run the same ten-second motion at different frame rates; travel and rotation remain close within a stated tolerance.
Pause and reset	All simulation motion/particles freeze. Camera/UI still work. Reset restores the documented starting state.
Gate and wheel limits	Gate stays within 0 1 and its slot. Linked flow, spray, wheel speed, and indicator agree.
Boat and water	Boat stays inside navigable water, follows the same wave height, and creates a speed-dependent wake.
Transform/normal checks	Parent motion carries children. A stretched object keeps sensible lighting. Mirrored winding is handled correctly.
Shading comparison	Same camera/light/mesh/material, visibly different Gouraud/Phong highlights, and labelled mode.
Ray intersection	Known hit, miss, behind-ray hit, nearer-hit choice, triangle-edge cases, and parallel box directions.
BVH and direct search	Same nearest hit and distance on a small deterministic ray set, within a stated numeric tolerance.
Shadow and reflection rays	Blocker removes direct light; point-light shadows stop at the light. Reflections show another object and terminate at the depth limit.
Snapshot matching	Moving gate/boat and water waves have the same frozen pose in raster and ray views. Stale work is cancelled.
Long running stability	Repeated detail changes, resets, and ray captures do not grow GPU buffers, particle pools, or snapshot memory indefinitely.
Resize and packaging	Aspect ratio stays correct; a fresh build finds its shaders and produces the documented scene.

Measure before optimising

Record interactive FPS, frame time, triangle/draw counts, and ray image render time separately. A practical initial target is at least 30 FPS in the normal view on the presentation computer; measure it there before claiming it. Keep a lower-detail preset for slower computers.

Reuse meshes, cache static terrain, bound particles, and avoid rebuilding static geometry every frame. Enable face culling on valid opaque meshes only; keep thin/double-sided panels and water intentional. Optimise the measured bottleneck rather than reducing important objects arbitrarily.

> **Done when**
>
> A short validation note contains the machine/settings, expected result, observed result, and screenshot or numeric evidence for each important check. Remaining visual/ray limitations are stated clearly.

Step 14 Prepare the teacher demonstration

Make the implementation easy to assess and explain.

A suggested six-minute demonstration

Time	Show and explain
0:00 0:40	Overview: identify the dam, two turbines, panels, reservoir, boat, and wheel. State the original graphics-focused proposal.
0:40 1:30	Operate wind and gate. Show the connected flow/wheel response. Steer the boat, then pause/resume.
1:30 2:10	Turbine/dam close-ups: show translation, rotation, pivot, scaling, and hierarchy. Open the relevant object file.
2:10 3:15	Gallery: switch Gouraud/Phong at fixed settings; isolate ambient/diffuse/specular; move the point light.
3:15 4:30	Frozen ray view: show camera-ray rendering, shadows on/off, reflections on/off, progress, and saved comparison images.
4:30 5:10	Day/night and simple rain; explain the particle pool and illustrative output bars.
5:10 6:00	Show file organisation, one step note, measured performance, and the known limitations.

What to submit

- Original proposal PDF plus a report describing the final implemented scope.
- Complete C++/GLSL source, support-library requirements, CMake configuration, and build/run instructions.
- A short README with controls, renderer modes, and known limitations.
- Step notes that explain the important maths and point to its files.
- Matching screenshots for Gouraud/Phong, lighting terms, daytime/night, gate closed/open, and ray shadows/reflections off/on.
- A validation/performance note and, if required by your course, a short recorded demonstration.

Do the final build from a clean directory

CMake currently gathers .cpp files only from core/ and objects/. Add the planned input/, systems/, rendering/, and ui/ sources explicitly or update source collection deliberately. Copy every required shader next to the executable and document its working directory. Build a fresh Release version and check launch, shader loading, controls, ray mode, and image saving.

> **Done when Milestone C**
>
> Another person can build/run the project using your README, operate the required controls, and repeat the demonstration. Every screenshot and report claim corresponds to a feature actually present in that build.

Your final checklist and reading notes

Use this page before calling the project complete.

Required finish check	Tick after demonstration
Proposal preserved	Same renewable energy scene and original proposal file.
Lighting	Sun, point light, material differences, and term controls work.
Transformations	Translation, rotation, scaling, composition, and hierarchy are demonstrated.
Gouraud and Phong	Correct separate calculations and a convincing matched comparison.
Motion	Two rotors, cloud, gate, boat, and wheel move continuously in demo mode.
Interaction	User operates wind/gate/boat and can pause, reset, inspect, and switch modes.
Ray tracing	Camera rays, geometry hits, shadow rays, and reflected rays create the displayed image.
Understandable source	Separate feature files, named settings, shared scene data, and short explanations.
Presentation quality	Readable default view/UI, saved evidence, measured performance, and repeatable presets.

Useful words

Mesh: vertices and triangles that describe an object. **Normal:** the direction a surface faces. **Shader:** a small GPU program. **Fragment:** a rasterized surface sample, usually contributing to a pixel. **Hierarchy:** parts following parent parts. **Rasterization:** projecting triangles onto the image. **Ray tracing:** finding surfaces by testing rays. **BVH:** a tree of boxes that helps rays skip triangles. **Snapshot:** one frozen scene state.

Optional work after the required version

Soft shadows, reflection antialiasing, Fresnel reflection, refraction, a live limited ray-shadow pass, battery/demand feedback, and an automated camera tour are sensible extensions. Add them only after the checklist above passes. Geometric mirroring, raster water reflection, and ray-traced reflection should be named separately.

Primary reading sources

[1] PBRT System Overview: ray rendering concepts and scene organisation.

[2] PBRT Triangle Meshes: ray/triangle intersections and surface data.

[3] PBRT Bounding Volume Hierarchies: accelerating ray tests.

[4] GLFW Input Guide: events, held keys, mouse input, and polling.

Local evidence: original 3D hybrid proposal; Confluence source/shaders; HOW_I_BUILT_THE_SCENE.md; IMPLEMENTATION_GUIDE.md; ship source, report, and README; Siyam s main.cpp, core modules, README, and proposal. External sources were checked on 8 October 2026.